САМАРСКИЙ УНИВЕРСИТЕТ
SAMARA UNIVERSITY

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра киберфотоники

Отчёт

Лабораторная работа №3

«Высокопроизводительные вычисления в механике»

Выполнил: студент группы 4446-010303D

Погудин М.Э.

Проверил: Неженский М. С.

«____» _____2026 г.

Оценка _____

Самара 2026

Для корректной работы программы на MPI необходимо переместить вызов MPI_Finalize() перед return 0 в функции main(). Функция MPI_Finalize() должна вызываться только один раз после завершения всех MPI-операций. Также необходимо раскомментировать строку //MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD), чтобы обеспечить корректную рассылку размера матрицы между процессами.

MPI использует ядра процессора напрямую, поэтому для данного эксперимента целесообразно ограничиться количеством процессов n = 4, где n - число задействованных ядер.

Компиляция программы выполняется с помощью команды:

mpicxx -o matMulMPI matMulMPI.cpp

Эксперименты проводились для n = 1, n = 2 и n = 4 на матрицах размером от 100 до 1000. Ниже приведен результат расчётов (см. Рисунок 1 – Рисунок 10).

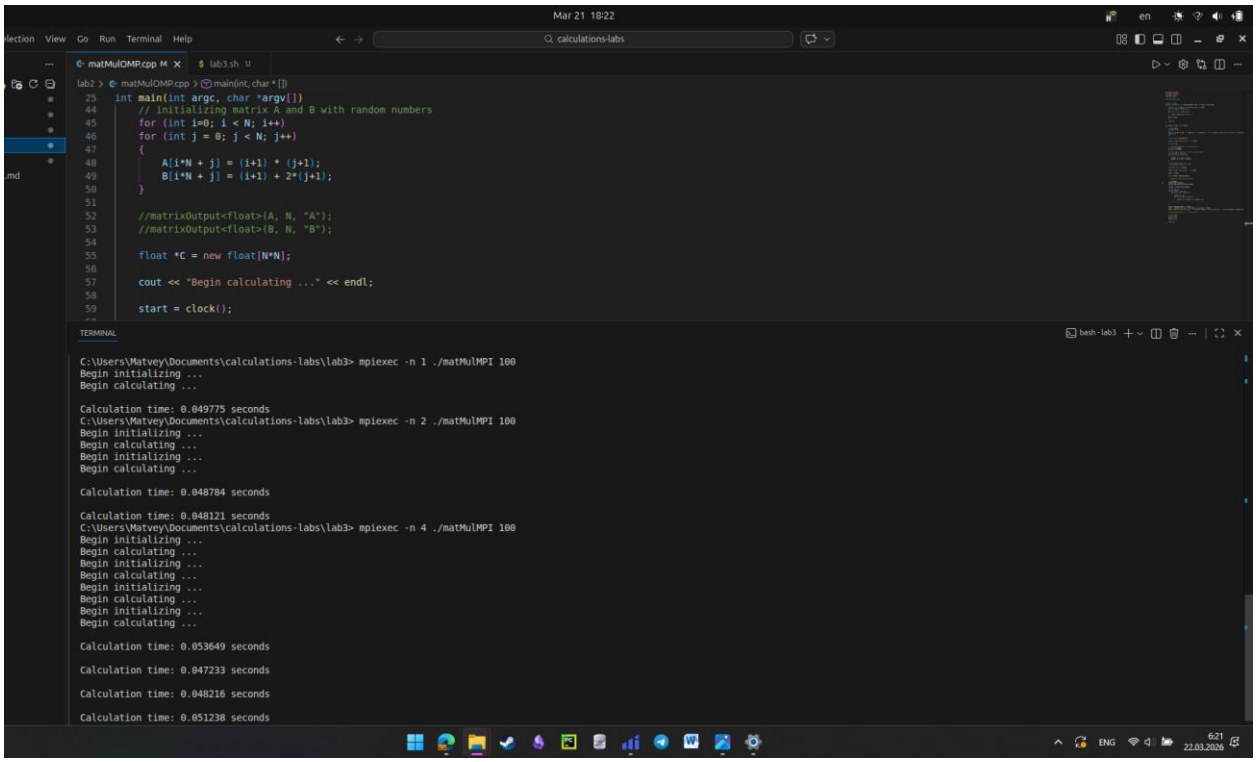

Рисунок 1 – Результаты вычислений

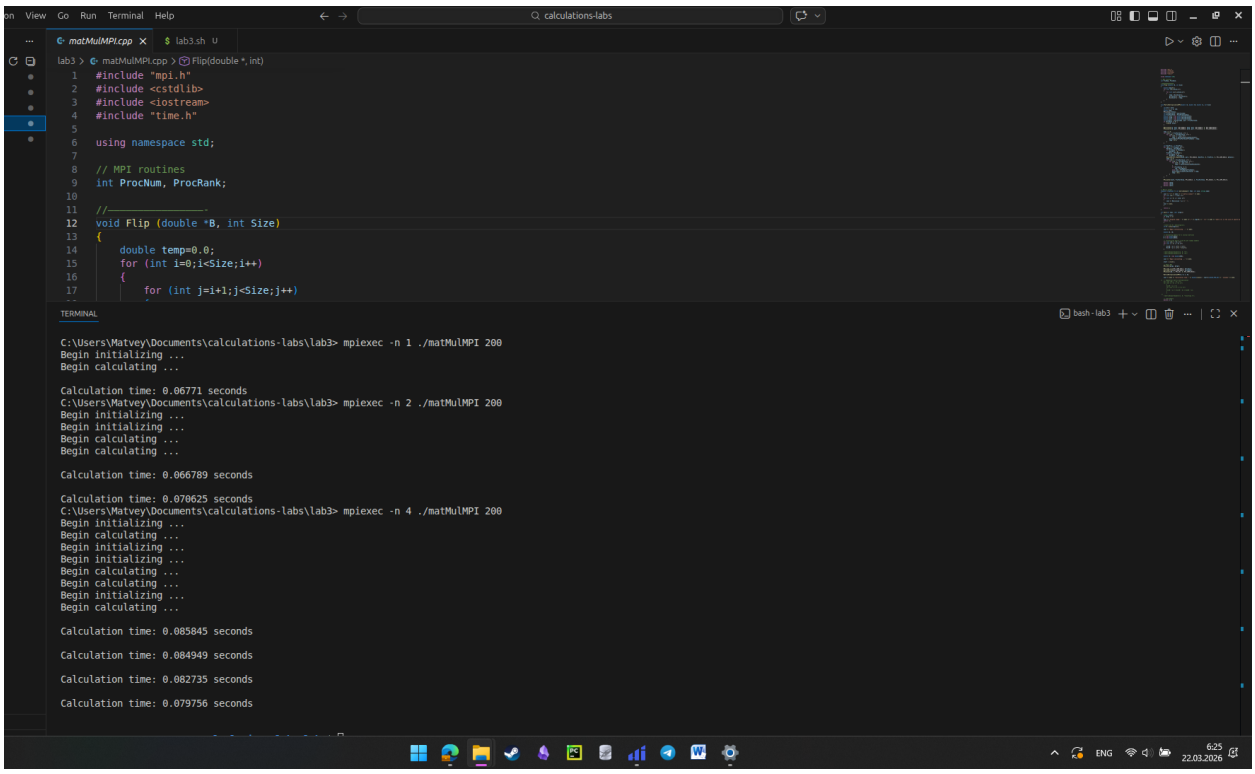

Рисунок 2 – Результаты вычислений

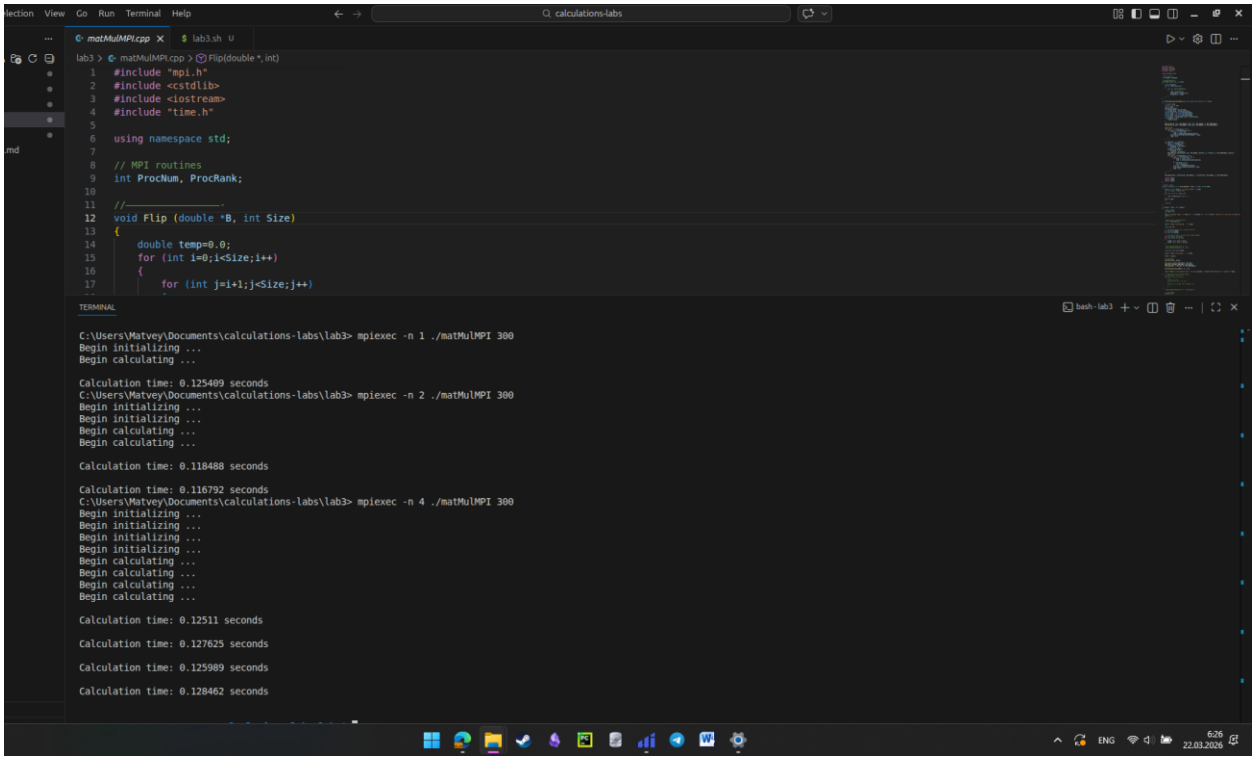

Рисунок 3 – Результаты вычислений

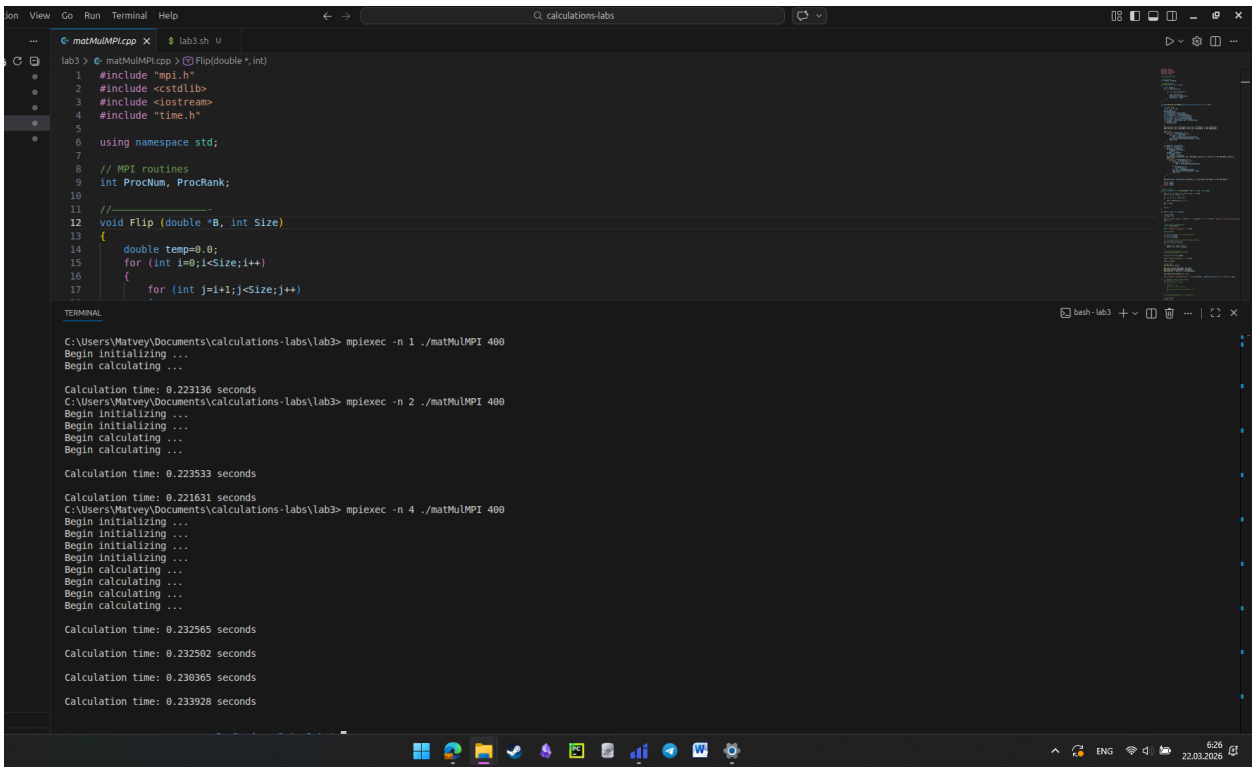

Рисунок 4 – Результаты вычислений

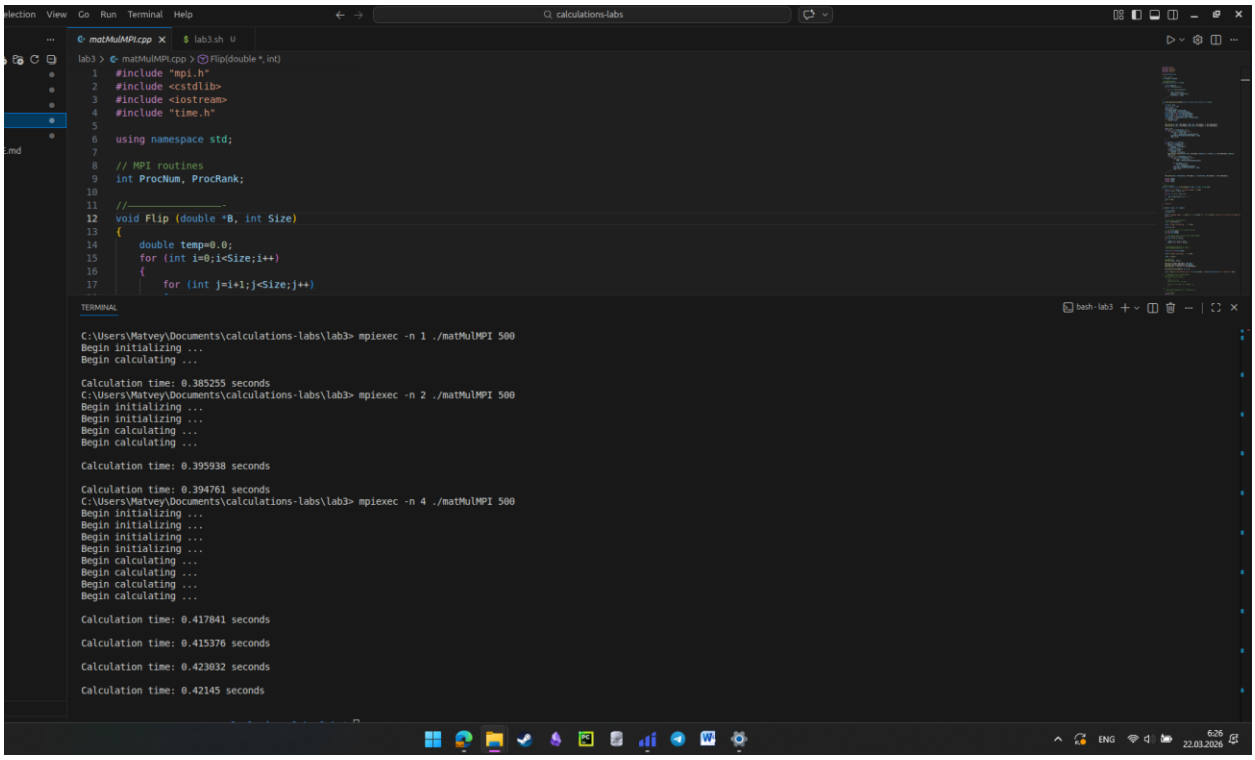

Рисунок 5 – Результаты вычислений

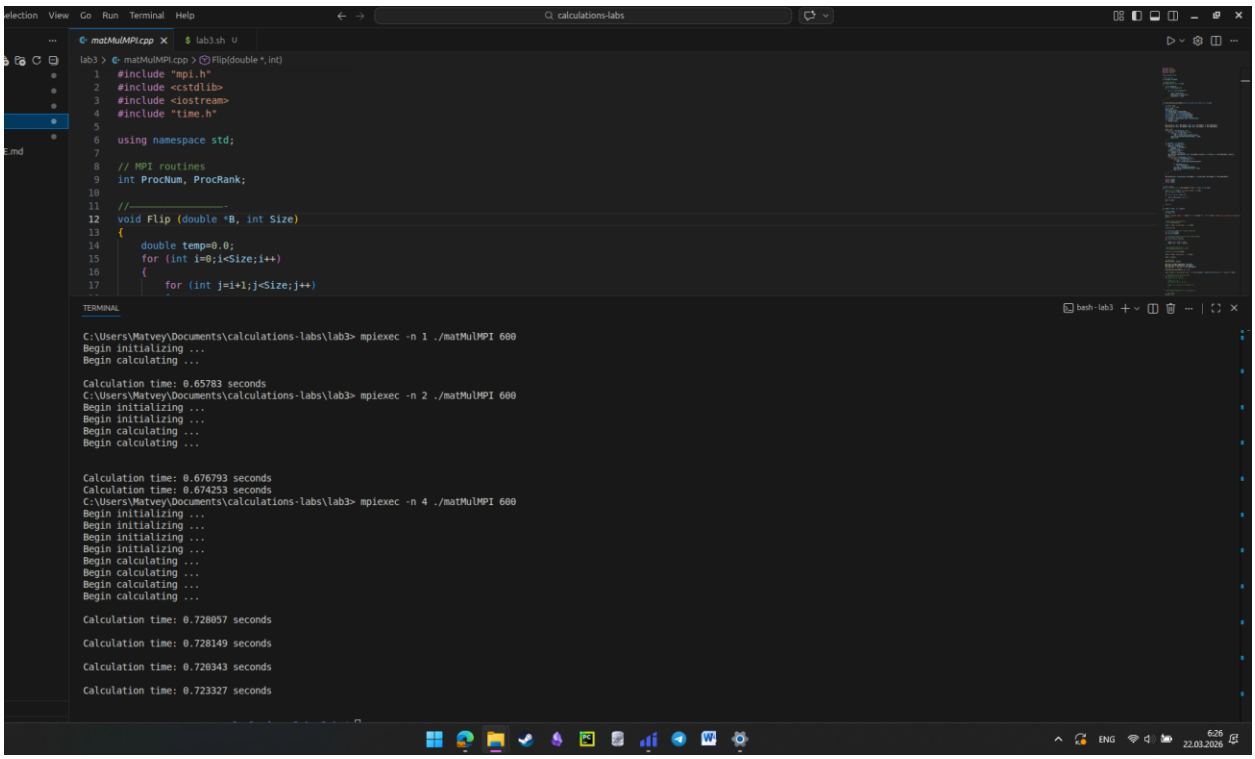

Рисунок 6 – Результаты вычислений

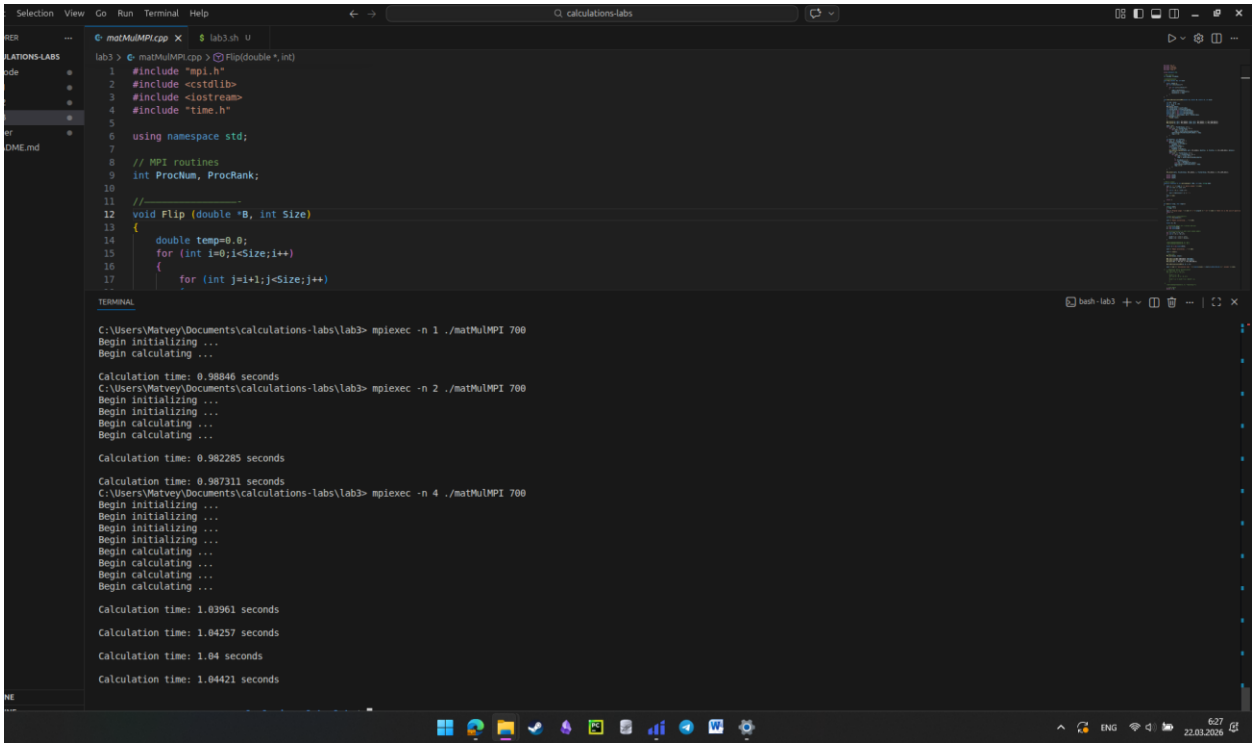

Рисунок 7 – Результаты вычислений

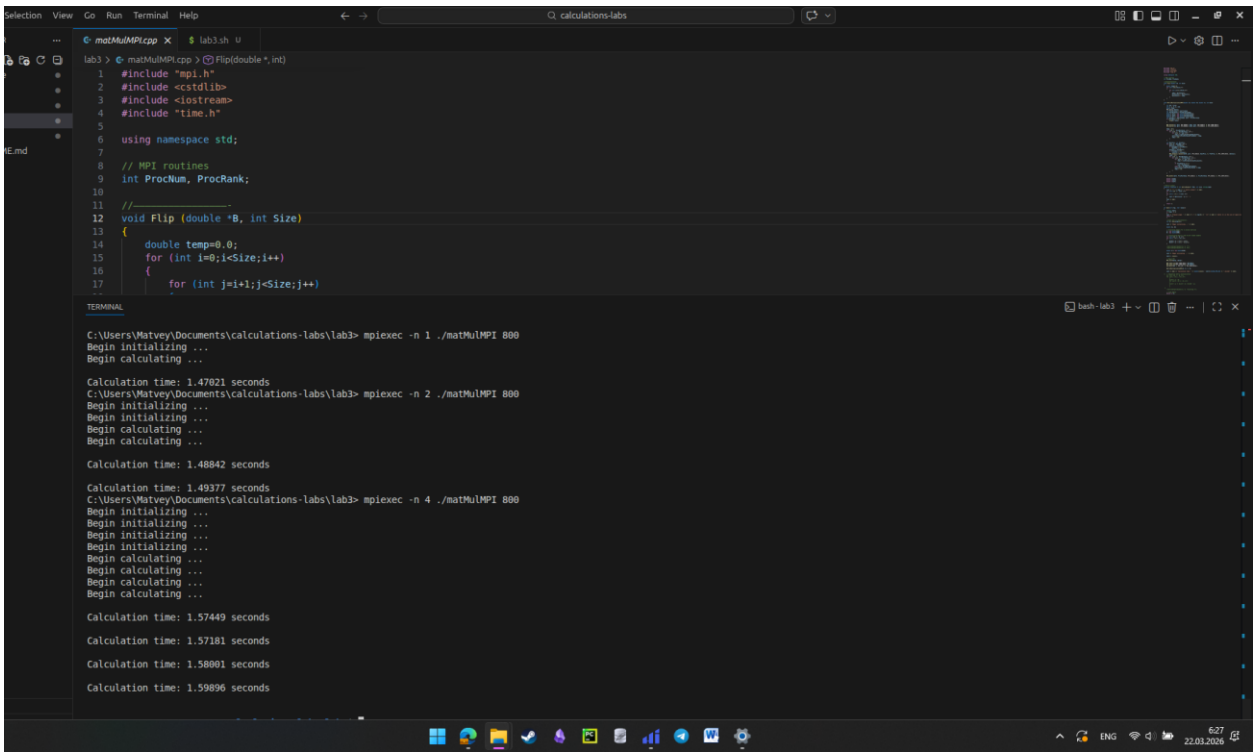

Рисунок 8 – Результаты вычислений

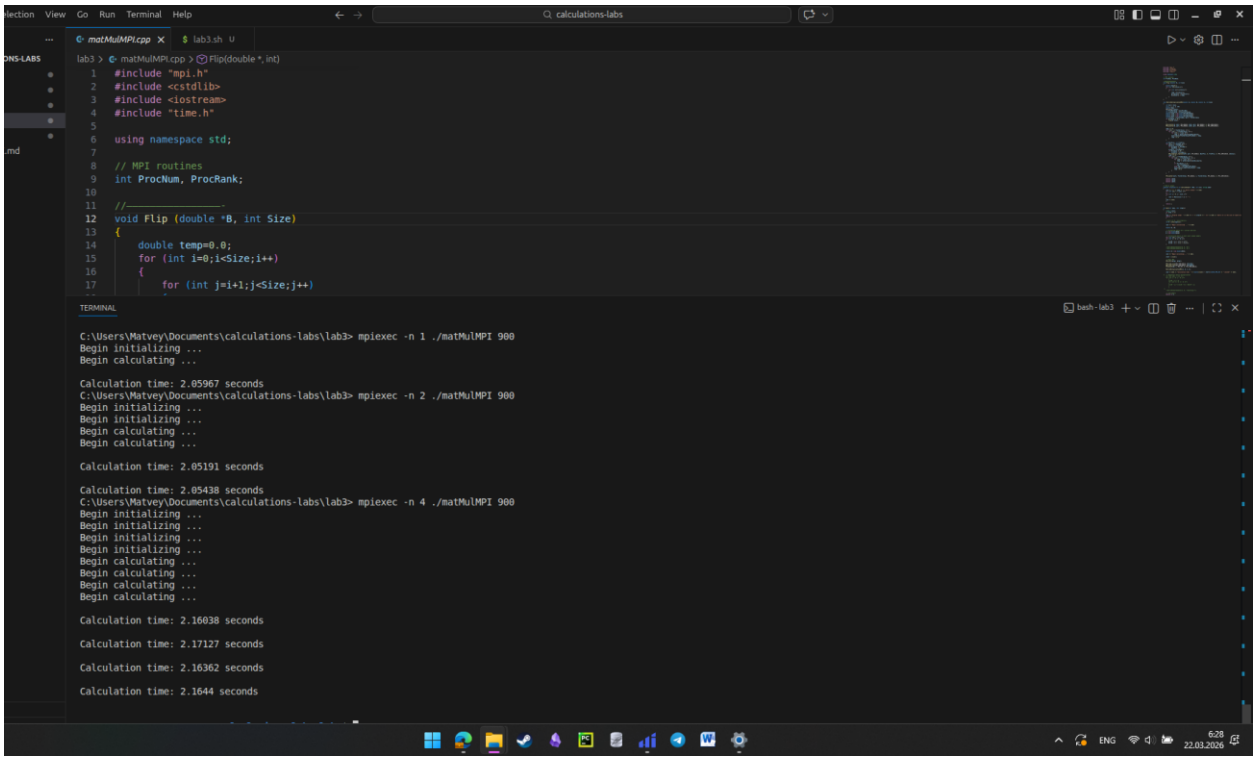

Рисунок 9 – Результаты вычислений

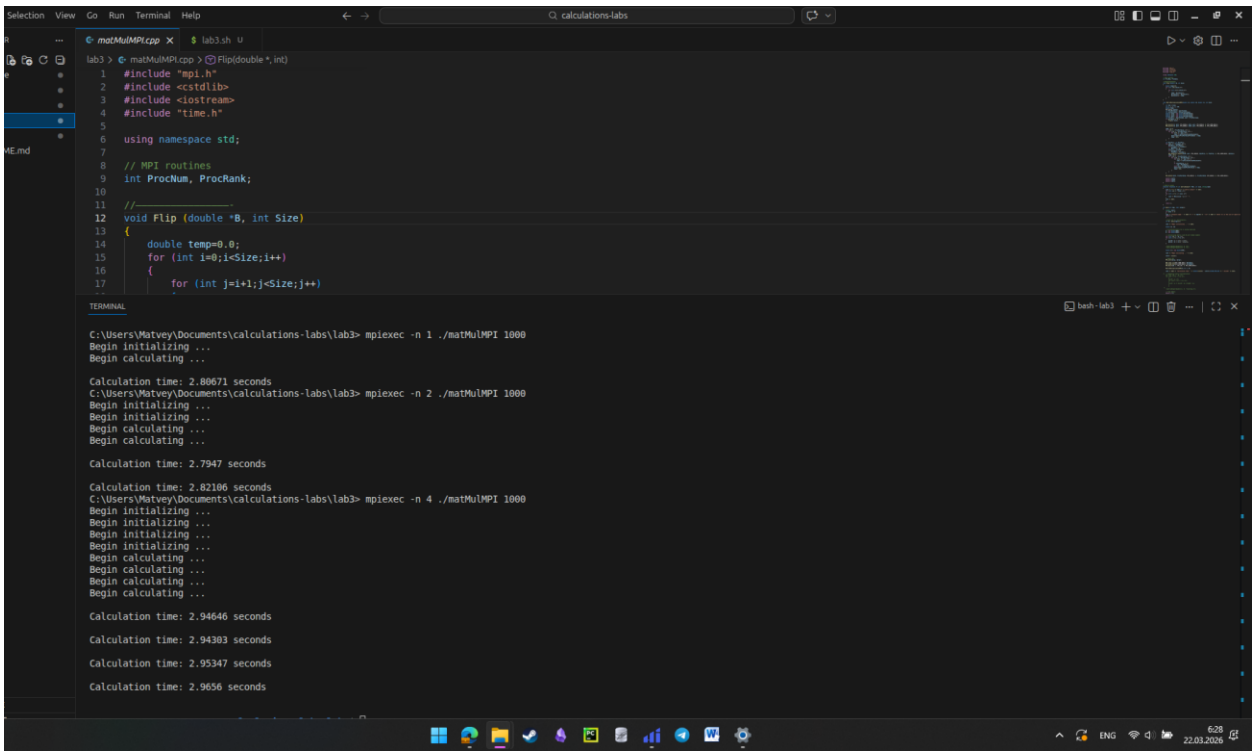

Рисунок 10 – Результаты вычислений

Из нескольких значений получим среднее и по результатам расчётов составим таблицу (см. Таблица 1) и построим график (см. Рисунок 11).

Таблица 1 – результаты расчётов

Размер матрицы n×n	Количество ядер		
	1 ядро	2 ядра	4 ядра
100	0,048086	0,0468835	0,0503198
200	0,067771	0,068707	0,0833213
300	0,125409	0,11764	0,1267965
400	0,223136	0,222582	0,2324113
500	0,385255	0,3953495	0,4194248
600	0,65783	0,675523	0,724969
700	0,98846	0,984798	1,0415975
800	1,47021	1,491095	1,5813175
900	2,05967	2,053145	2,1649175
1000	2,80671	2,80788	2,95214

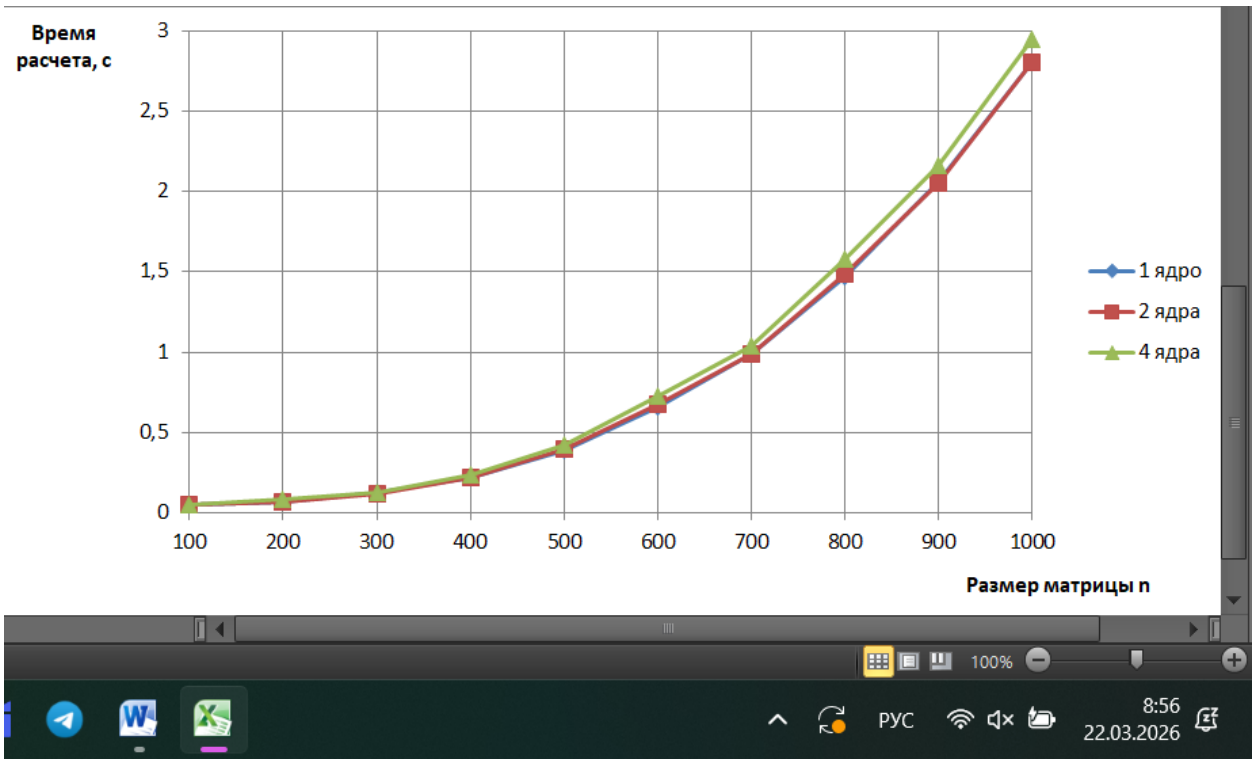

Рисунок 11 – График времени вычислений

Приведем сравнения времени при разных видах расчёта.

Таблица 2 – Сравнение всех результатов расчётов

Размер матриц n×n	Прямой метод	Количество потоков						Количество ядер		
		1 поток	2 потока	4 потока	8 потоков	16 потоков	32 потока	1 ядро	2 ядра	4 ядпра
500	0.29	-	-	-	-	-	-	0,048086	0,0468835	0,0503198
600	0.50	-	-	-	-	-	-	0,067771	0,068707	0,0833213
700	0.84	-	-	-	-	-	-	0,125409	0,11764	0,1267965
800	1.26	-	-	-	-	-	-	0,223136	0,222582	0,2324113
900	1.86	-	-	-	-	-	-	0,385255	0,3953495	0,4194248
1000	2.67	3,41	2,4	2,3	2,6	2,9	3,2	0,65783	0,675523	0,724969
1100	3.49	-	-	-	-	-	-	0,98846	0,984798	1,0415975

По полученным результатам можем привести сравнение с результатами прямого метода и метода с применением разного количества потоков.

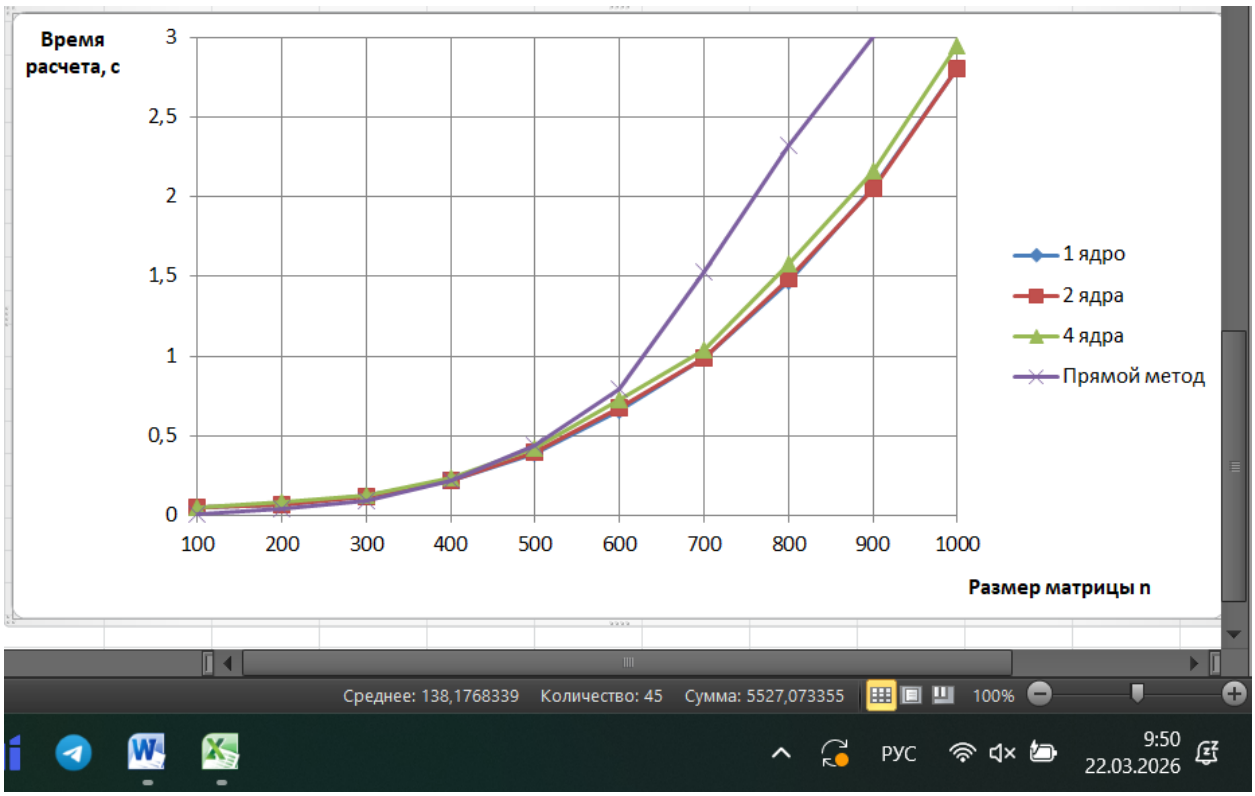

Рисунок 12 – Сравнительный график результатов вычислений прямым методом и при разном количестве ядер

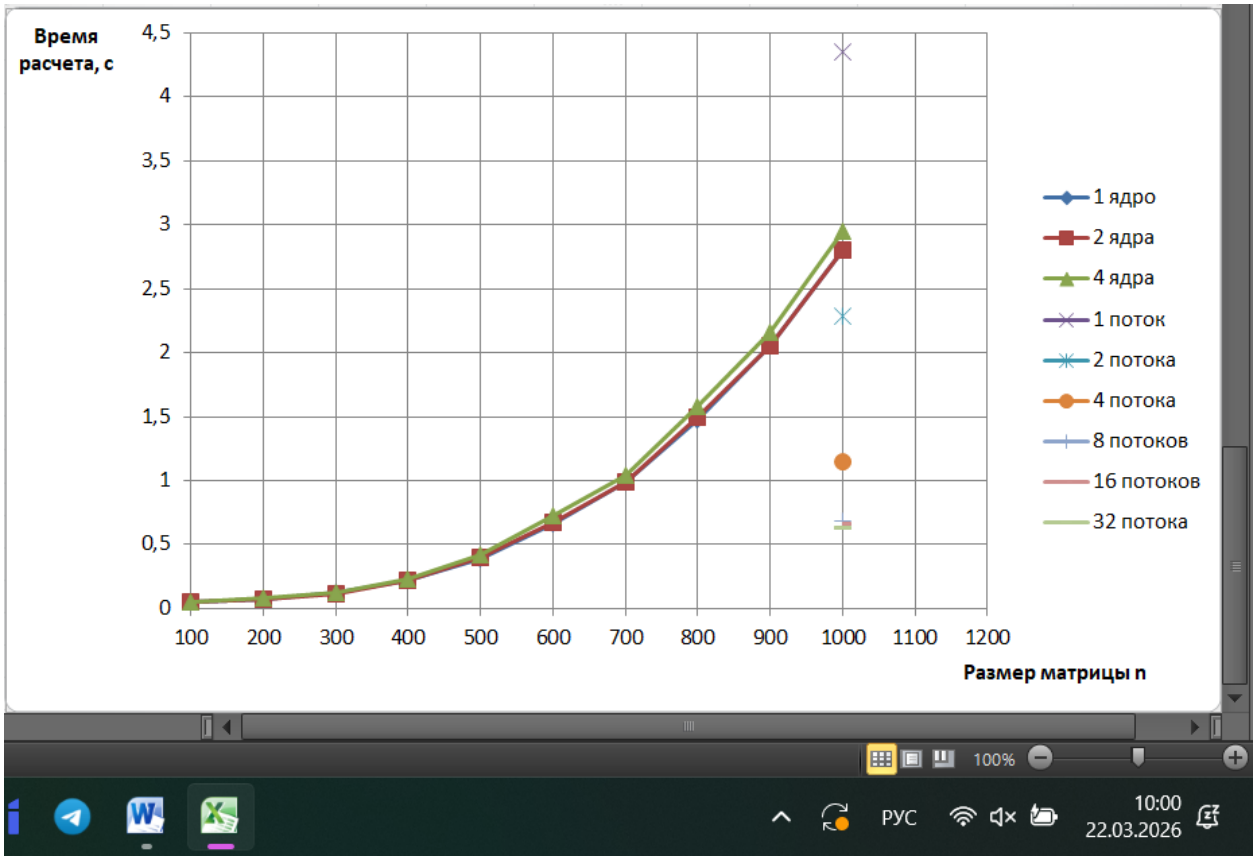

Рисунок 13 – Сравнительный график для метода с разным количеством потоков и разным количеством ядер

Вывод

По результатам измерений времени расчета можно сделать вывод, что изменение количества ядер процессора не влияет на время выполнения одного вычисления, однако позволяет выполнять несколько вычислений параллельно, повышая общую производительность системы.

При сравнении прямого метода с параллельным методом на разном количестве ядер наблюдается заметная разница в скорости вычислений для матриц размером более 600×600. Параллельный подход демонстрирует существенное преимущество в таких случаях.

Для матрицы размером 1000×1000 сравнение методов с разным количеством ядер и разным числом потоков выявило оптимальный режим работы. При использовании 2 и более потоков скорость вычисления значительно выше по сравнению с использованием только разного числа ядер без оптимизации потоков.

Приложение

Ссылка на репозиторий с выполненными лабораторными работами: https://github.com/MatveyPogudin/high-performance-computing